

Unit 1

2 Marks

1. ****What is an Algorithm? What are the criteria for writing an algorithm?****

Ans: - An algorithm is a step-by-step set of instructions for solving a particular problem or performing a specific task. It's a precise and unambiguous sequence of operations that, when executed, produces a desired result.

Criteria for writing an algorithm:

- ****Input****: It should specify what data is needed as input.
- ****Output****: It should define the expected result or output.
- ****Finiteness****: The algorithm must terminate after a finite number of steps.
- ****Definiteness****: Each step must be precisely defined and unambiguous.
- ****Effectiveness****: Every step should be executable and achieve a specific task.
- ****Generality****: It should be applicable to a range of input values or instances.

2. **What are the methods of specifying an algorithm?**

Ans: - Algorithms can be specified using various methods, including:

- Natural Language (e.g., English)
- Pseudocode
- Flowcharts
- Programming languages
- Decision tables
- State diagrams

3. **List the steps of Algorithm design and analysis process:**

Ans: - Problem definition

- Design a solution (algorithm)
- Analysis of the algorithm
- Implementation
- Testing and debugging
- Documentation

4. **What is an exact algorithm and approximation algorithm? Give an example:**

Ans: - An exact algorithm is one that guarantees to find the optimal solution to a problem. It provides the most accurate and precise result.

- An approximation algorithm is a heuristic method that finds a solution that is not necessarily optimal but is close to the optimal solution. It is often used for complex problems when finding an exact solution is computationally infeasible.

Example: - Exact algorithm: Solving the traveling salesman problem to find the shortest route that visits a set of cities exactly once.

- Approximation algorithm: The nearest neighbor algorithm for the traveling salesman problem, which finds a good but not necessarily optimal solution.

5. List the important Problem Types:

Ans:

- Sorting problems
- Searching problems
- Optimization problems
- Combinatorial problems
- Graph problems
- Geometric problems
- Network flow problems
- Decision problems

6. Define the different methods for measuring algorithm efficiency:

Ans:

- Time complexity: Measures the time an algorithm takes to execute as a function of the input size.
- Space complexity: Measures the memory space used by an algorithm in relation to the input size.
- Computational complexity: Evaluates an algorithm's performance under different resource constraints.
- Asymptotic analysis: Analyzes algorithm efficiency as input size approaches infinity.

7. Write the Euclid algorithm to find the GCD of 2 numbers:

Ans: Python code:

```
def euclid_gcd(a, b):  
    while b:  
        a, b = b, a % b  
    return a
```

8. What are combinatorial problems? Give an example:

Ans: - Combinatorial problems involve counting, arranging, or selecting objects in a specific way. They often deal with discrete, finite sets of elements.

Example: The traveling salesman problem, where you need to find the shortest route that visits a set of cities exactly once.

9. Define the following data structures:

- a) Single linked list:** A data structure in which each element points to the next one, forming a linear sequence.
- b) Double linked list:** Similar to a single linked list, but each element has pointers to both the next and previous elements.
- c) Stack:** A linear data structure that follows the Last-In-First-Out (LIFO) principle.
- d) Queue:** A linear data structure that follows the First-In-First-Out (FIFO) principle.
- e) Graph:** A collection of nodes (vertices) and edges that connect pairs of nodes.
- f) Tree:** A hierarchical data structure with a single root node and child nodes, which are organized in a branching structure.

10. Explain the terms (w.r.t graph):

a) Directed graph: A graph in which edges have a direction, meaning they go from one node to another (arcs).

b) Undirected graph: A graph in which edges have no direction, and they simply connect nodes (edges).

c) Adjacency matrix: A matrix representation of a graph where rows and columns correspond to nodes, and the entries indicate whether there is an edge between nodes.

d) Adjacency lists: A representation of a graph where each node maintains a list of its adjacent nodes.

e) Weighted graph: A graph where edges have associated weights or costs.

f) Path: A sequence of nodes in a graph where each consecutive pair is connected by an edge.

g) Cycle: A path in a graph that starts and ends at the same node, visiting at least one other node in between.

11. Explain the terms (w.r.t trees):

a) Free tree: A tree with no specific root node.

b) Forest: A collection of disjoint trees.

c) Rooted tree: A tree with a designated root node from which all other nodes are reachable.

d) Ordered tree: A tree where the children of each node are ordered, and the order matters.

e) Binary search tree: A binary tree where the left subtree of a node contains values less than the node, and the right subtree contains values greater than the node.

12. Define Sets and Dictionaries:

Ans: - Sets: Sets are collections of unique elements with no specific order. In mathematics, sets are represented using curly braces, e.g., {1, 2, 3}.

- Dictionaries: Dictionaries (also known as maps or associative arrays) are data structures that store key-value pairs. Each key is associated with a value, allowing for efficient retrieval of values based on their keys.

13. Define the two types of efficiencies used in algorithms:

Ans: - Time efficiency: Measures the time it takes for an algorithm to complete a task based on input size.

- Space efficiency: Measures the memory space used by an algorithm to store data structures and variables.

14. What are Best case and Worst case in algorithm:

Ans: - Best case: It refers to the scenario in which an algorithm performs optimally with the least possible time or resource usage. It represents the lower bound on the algorithm's performance.

- Worst case: It refers to the scenario in which an algorithm performs with the maximum possible time or resource usage. It represents the upper bound on the algorithm's performance.

15. Why is order growth necessary in algorithm analysis:

Ans:- Order growth (Big O notation) is necessary for algorithm analysis because it provides a way to describe the behavior of an algorithm as the input size grows. It allows us to understand how the algorithm's performance scales with larger inputs and make informed decisions about choosing the right algorithm for a specific problem. Order growth analysis helps in comparing and selecting algorithms based on their efficiency.

16. What are asymptotic notations? Why are they required:

Ans: - Asymptotic notations are mathematical notations used to describe the limiting behaviour of functions as their input values approach infinity. They are required for algorithm analysis to provide a concise and standardized way of expressing the upper and lower bounds of algorithm performance, helping us compare algorithms without getting lost in specific constant factors.

17. What is Big O notation? Give an example:

Ans: - Big O notation, often denoted as $O(f(n))$, describes the upper bound on the growth rate of a function, indicating the worst-case time complexity of an algorithm. It provides an asymptotic upper limit.

Example: $O(n^2)$ represents a quadratic time complexity, which means an algorithm's running time grows no faster than the square of the input size.

18. What is Big Omega notation? Give an example:

Ans: - Big Omega notation, denoted as $\Omega(f(n))$, describes the lower bound on the growth rate of a function, indicating the best-case time complexity of an algorithm. It provides an asymptotic lower limit.

Example: $\Omega(n)$ represents a linear time complexity, which means an algorithm's running time grows at least as fast as the input size.

19. Define Big Theta notation. Give an example:

Ans - Big Theta notation, denoted as $\Theta(f(n))$, describes both the upper and lower bounds on the growth rate of a function, indicating the tightest possible bound on an algorithm's time complexity. Example: $\Theta(n)$ represents linear time complexity, meaning an algorithm's running time grows exactly in proportion to the input size.

20. Define Little Oh notation. Give an example:

Ans: - Little Oh notation, denoted as $o(f(n))$, describes a growth rate strictly less than a function, indicating that an algorithm's performance is better than the specified function. Example: If an algorithm has a time complexity of $o(n)$, it means the algorithm is faster-growing than linear (i.e., better than linear).

21. What is a recurrence relation? Give an example:

Ans: - A recurrence relation is a mathematical equation or formula that expresses a function's value in terms of one or more of its previous values. It is often used to describe the time complexity of recursive algorithms.

Example: The Fibonacci sequence is described by the recurrence relation $F(n) = F(n-1) + F(n-2)$ with initial values $F(0) = 0$ and $F(1) = 1$, where $F(n)$ represents the n th Fibonacci number.

22. Prove the following statements:

a) $100n + 5 = O(n^2)$ - True, because it is a linear function, and n^2 is an upper bound.

b) $n^2 + 5n + 7 = \Theta(n^2)$ - True, as it matches the tightest bound for a quadratic function.

c) $n^2 + n = O(n^3)$ - True, because it is an upper bound and n^3 grows faster than n^2 .

d) $\frac{1}{2}n(n-1) = \Theta(n^2)$ - True, as it is a quadratic function and matches both upper and lower bounds.

e) $5n^2 + 3n + 20 = O(n^2)$ - True, as it is a quadratic function, and n^2 is an upper bound.

f) $\frac{1}{2}n^2 + 3n = \Theta(n^2)$ - True, as it is a quadratic function and matches both upper and lower bounds.

g) $n^3 + 4n^2 = \Omega(n^2)$ - True, because it is a lower bound, and n^2 is a lower bound as well.

23. Algorithm Sum(n):

S <- 0

For i <- 1 to n do

S <- **S** + i

Return **S**

a) What does this algorithm compute?

Ans: - This algorithm computes the sum of all integers from 1 to n .

b) What is its basic operation?

Ans: - The basic operation is the addition operation ($S = S + i$).

c) How many times is the basic operation executed?

Ans: - The basic operation is executed ' n ' times since it's inside a loop that runs from 1 to ' n '.

d) What is the efficiency class of this algorithm?

Ans: - The efficiency class of this algorithm is $O(n)$ because it has a linear time complexity, and the number of basic operations is directly proportional to the input size ' n '.

3 or More marks

1. What is an Algorithm? Explain the various criteria for writing an algorithms with example?

Ans: An algorithm is a step-by-step set of instructions or rules designed to solve a specific problem or perform a particular task.

An algorithm is a sequence of instructions that a computer must perform to solve a well-defined problem. It essentially defines what the computer needs to do and how to do it. Algorithms can instruct a computer how to perform a calculation, process data, or make a decision.

Criteria for Writing Algorithms:

1. Input: The algorithm must have well-defined inputs, specifying the data or information it operates on.
2. Output: There should be a clear definition of what the algorithm produces as a result or output.
3. Definiteness: Each step of the algorithm must be clear, unambiguous, and executable.
4. Finiteness: The algorithm should terminate after a finite number of steps, avoiding infinite loops.
5. Effectiveness: Every instruction in the algorithm should be basic and executable.

Example: Finding the Maximum Number in a List

```
```python
```

Algorithm Find Max(List):

Input: List of numbers

Output: Maximum number in the list

1. Set max to the first element of the list.
2. For each element in the list:
  - a. If the current element is greater than max, update max.
3. Return max.

```
```
```

2. Explain Euclid Algorithm with example to find the GCD of two numbers.

Euclid's Algorithm:

Ans: Euclid's algorithm is a method for computing the greatest common divisor (GCD) of two numbers. The algorithm is based on the principle that the GCD of two numbers also divides their difference. Here's how it works:

1. Given two numbers a and b , where $a > b$, you subtract b from a until the result is less than b .
2. Then, you swap a and b , and repeat the process until a becomes 0.
3. The GCD is the last non-zero remainder.

For example, let's find the GCD of 48 and 18:

Step 1: Subtract 18 from 48 to get 30.

Step 2: Subtract 18 from 30 to get 12.

Step 3: Now, 12 is less than 18, so swap 12 and 18 to get $a = 18$ and $b = 12$.

Step 4: Subtract 12 from 18 to get 6.

Step 5: Now, 6 is less than 12, so swap 6 and 12 to get $a = 12$ and $b = 6$.

Step 6: Subtract 6 from 12 to get 6.

Step 7: Now, 6 equals 6, so $a = 6$ and $b = 6$.

Step 8: Subtract 6 from 6 to get 0.

So, the GCD of 48 and 18 is 6.

3. Explain Consecutive integer checking methods of find the GCD of two numbers.

Ans: Consecutive Integer Checking Algorithm:

The Consecutive Integer Checking Algorithm is a simple, but inefficient, method to find the GCD of two numbers. Here's how it works:

1. Start with the smaller of the two numbers.
2. Check if it divides both numbers. If it does, it's the GCD.
3. If not, subtract one and try again.
4. Repeat until you find a number that divides both numbers evenly.

For example, let's find the GCD of 48 and 18:

- Start with 18 (the smaller number).
- Check if 18 divides both 48 and 18. It doesn't ($48 / 18$ leaves a remainder), so subtract 1 to get 17.
- 17 doesn't divide both numbers, so subtract 1 to get 16.
- Continue this process until you reach 6.
- 6 divides both 48 and 18, so the GCD of 48 and 18 is 6.

This method is straightforward, but it can be slow for large numbers because it potentially requires checking all integers down to 1. Euclid's algorithm, on the other hand, is much faster and more efficient for large numbers.

4. Explain Algorithm design and analysis process with flow diagram.

Ans: Algorithm Design and Analysis Process:

****Algorithm Design and Analysis Process:****

1. Problem Definition: Clearly define the problem that the algorithm aims to solve.
2. Algorithm Construction: Develop the algorithm using suitable methods (pseudocode, flowcharts, etc.).
3. Verification: Check the algorithm for correctness and accuracy.
4. Analysis: Evaluate the algorithm's efficiency, time complexity, & space complexity.
5. Optimization: Modify the algorithm to improve efficiency if necessary.
6. Documentation: Provide clear documentation for future reference.

****Flow Diagram:****

...

[Problem Definition] --> [Algorithm Construction] --> [Verification]
--> [Analysis] --> [Optimization] --> [Documentation]

5. Explain any FIVE Problem types.

Ans: Five Problem Types:

1. **Sorting Problems:** Involving the arrangement of elements in a specific order. The sorting problem involves rearranging the items of a given list in a specific order (usually non-decreasing). This is a fundamental operation in many applications, including ranking search results, data analysis, and preparing data for other algorithms. For example, sorting algorithms include QuickSort, MergeSort, and BubbleSort.
 2. **Searching Problems:** Finding a particular element in a set of data. Searching involves finding a specific item in a data structure. This is a key operation in many applications, such as looking up a contact in a phone book or finding a webpage in a search engine's index. Examples of searching algorithms include Binary Search and Linear Search.
 3. **String Processing:** String processing problems involve manipulation and analysis of strings. This includes operations like searching for patterns in text, comparing strings, and transforming one string into another. Examples of algorithms used in string processing include the Knuth-Morris-Pratt (KMP) algorithm for pattern searching and the Levenshtein distance algorithm for measuring the difference between two strings.
 4. **Graph Problems:** Dealing with graphs and networks. Graph problems involve the study of graphs, which are mathematical structures used to model pairwise relations between objects. Examples of graph problems include finding the shortest path between two nodes, determining whether a graph is connected, and finding a cycle in a graph. Algorithms used to solve these problems include Dijkstra's algorithm for shortest paths and Kruskal's algorithm for minimum spanning trees.
 5. **Combinatorial Problems:** Involving arrangements and combinations.
 6. **Geometric Problems:** Related to geometry and spatial relationships.
- These problem types encompass a wide range of challenges that algorithms are designed to address, showcasing the diverse applications of algorithmic solutions in various domains.

6. Explain following

a. Graph problem

b. Combinatorial problems

c. Geometrical problems.

Ans: a. Graph Problem:

A graph problem involves the study and manipulation of graphs, which are mathematical structures representing relationships between pairs of objects. Graph problems can include finding paths, determining connectivity, and analyzing network structures.

b. Combinatorial Problems:

Combinatorial problems deal with counting, arranging, and selecting objects. These problems often involve combinations, permutations, and arrangements of elements. Examples include the traveling salesman problem and the assignment problem.

c. Geometrical Problems:

Geometrical problems relate to spatial and geometric relationships. These problems often involve finding measurements, angles, areas, or volumes in a geometric context. Examples include finding the area of a polygon or determining the distance between two points in space.

7. Explain the Fundamentals of Data Structure:

Ans: some of the fundamental data structures used in algorithms:

1. ****Array****: An array is a versatile and dynamic list that stores a collection of items. The items can be of the same type and are stored in contiguous memory locations³.
2. ****Linked List****: A linked list is a linear data structure where each element is a separate object. Each element (node) of a list is comprising of two items - the data and a reference to the next node³.
3. ****Hash Table****: A hash table, also known as a hash map, is a data structure that implements an associative array abstract data type, a structure that can map keys to values³.
4. ****Stack****: A stack is a linear data structure that follows a particular order in which the operations are performed. The order may be LIFO (Last In First Out) or FILO (First In Last Out)³.
5. ****Queue****: A queue is a kind of abstract data type or collection in which the entities in the collection are kept in order and the principal (or only) operations on the collection are the addition of entities to the rear terminal position, known as enqueue, and removal of entities from the front terminal position, known as dequeue³.
6. ****Heap****: A Heap is a special Tree-based data structure in which the tree is a complete binary tree³.
7. ****Binary Search Tree****: A Binary Search Tree (BST) is a tree in which all the nodes follow the below property: The left sub-tree of a node has a key less than or equal to its parent node's key. The right sub-tree of a node has a key greater than to its parent node's key³.

These data structures are fundamental because they are used in a wide variety of algorithms and can be used to solve a wide range of problems. Each data structure has its own strengths and weaknesses, and the choice of which data structure to use depends on the specific constraints of the problem you're trying to solve.

8. Write a note on Graph Data Structure:

Ans: Graph:- A collection of nodes (vertices) and edges connecting pairs of nodes.

A Graph is a non-linear data structure that consists of vertices (or nodes) and edges.

The edges are lines or arcs that connect any two nodes in the graph. More formally, a Graph is composed of a set of vertices (V) and a set of edges (E), and is denoted by G (E, V).

key components and concepts related to Graphs:

- **Vertices:** Vertices are the fundamental units of the graph. Sometimes, vertices are also known as vertex or nodes.
- **Edges:** Edges are drawn or used to connect two nodes of the graph. It can be an ordered pair of nodes in a directed graph.
- **Directed and Undirected Graphs:** In a directed graph, edges form an ordered pair. Edges represent a specific path from some vertex A to another vertex B. In an undirected graph, edges are not associated with directions².
- **Weighted Graph:** In a weighted graph, each edge is assigned with some data such as length or weight.
- **Degree of the Node:** A degree of a node is the number of edges that are connected with that node.
- **Graphs are used to solve many real-life problems.** They are used to represent networks, which may include paths in a city or telephone network or circuit network. Graphs are also used in social networks like LinkedIn, Facebook.

9. Write a note on following data structures.

a. Tree

b. Sets

c. Dictionary

Ans: a. ****Tree**:** A tree is a non-linear data structure that consists of nodes connected by edges⁴. It has a hierarchical relationship between the nodes⁴. The tree data structure stems from a single node called a root node and has subtrees connected to the root⁸. Examples of non-linear data structures are trees and graphs².

b. ****Sets**:** A set is a data structure that stores a collection of unique elements, with no duplicates allowed¹. Sets can be implemented using a variety of data structures, including arrays, linked lists, binary search trees, and hash tables¹. The various operations that are performed on a set are add or insert, replace or reassign, delete or remove, find or lookup¹.

c. ****Dictionary**:** A dictionary is a general-purpose data structure for storing a group of objects⁹. A dictionary has a set of keys and each key has a single associated value⁹. When presented with a key, the dictionary will return the associated value⁹. The various operations that are performed on a dictionary are add or insert, replace or reassign, delete or remove, find or lookup⁹.

10. Explain Space complexity and Time complexity with example.

Ans: 1. ****Time Complexity****: Time complexity of an algorithm quantifies the amount of time taken by an algorithm to run as a function of the length of the input¹. It is the time needed for the completion of an algorithm¹.

For example, consider the problem of finding whether a pair (X, Y) exists in an array, A of N elements whose sum is Z. The simplest idea is to consider every pair and check if it satisfies the given condition or not¹. The pseudo-code is as follows:

```
```python
int a [n]
for (int i = 0; i < n; i++)
 cin >> a [i]
for (int i = 0; i < n; i++)
 for (int j = 0; j < n; j++)
 if (i != j && a [i] + a [j] == z)
 return true
return false
```

Assuming that each of the operations in the computer takes approximately constant time, let it be c. The number of lines of code executed actually depends on the value of Z<sup>1</sup>.

2. **\*\*Space Complexity\*\***: Space complexity is a parallel concept to time complexity. It is the amount of memory space that an algorithm needs to run to completion<sup>4</sup>. The space needed by an algorithm increases with the size of the input<sup>4</sup>.

For example, if we need to create an array of size n, this will require O(n) space. If we create a two-dimensional array of size n\*n, this will require O(n<sup>2</sup>) space<sup>4</sup>. In recursive calls stack space also counts<sup>4</sup>.

These complexities are fundamental to the study of algorithms, as they provide a measure of the efficiency of an algorithm. An algorithm that has good time and space complexity is considered efficient<sup>1234</sup>.

## 11. Write an algorithm find sum of two matrixes also calculate its time complexity.

**Ans:** ```python

Algorithm AddMatrices(A, B):

Input: Matrices A and B of size m x n

Output: Resultant matrix C of size m x n

1. Initialize an empty matrix C of size m x n.

2. For each row i from 1 to m:

a. For each column j from 1 to n:

i. Set C[i][j] = A[i][j] + B[i][j].

3. Return matrix C.

```

Time Complexity: ($O(m*n)$), where m is the number of rows and n is the number of columns in the matrices A and B .

12. Write an algorithm find the sum of n numbers also calculate its space and time complexity.

Ans: algorithm in Python to find the sum of n numbers:

```
```python
def find_sum(n):
 total = 0
 for i in range(1, n+1):
 total += i
 return total
```
```

In this algorithm, the time complexity is $O(n)$ because there is a loop that runs n times¹. The space complexity is $O(1)$ because only a constant amount of space is used (for the variables $total$ and i)¹.

13. Explain the following w.r.t algorithm efficiency.

a. Measuring input size

b. Unit for measuring the run time

c. Order growth

- a. ****Measuring input size****: The input size is typically represented by ' n '. It is the number of elements that the algorithm operates on. For example, in a sorting algorithm, ' n ' would represent the number of elements in the list to be sorted⁹[¹⁰]¹¹².
- b. ****Unit for measuring the run time****: The run time of an algorithm is usually measured in terms of the number of operations performed, not in actual time units like seconds. This is because the actual time can vary depending on the machine, compiler, etc. The unit for measuring the run time is often represented using Big O notation¹⁷¹⁸¹⁹[²⁰].
- c. ****Order growth****: The order of growth of an algorithm, often represented using Big O notation, describes how the run time of an algorithm increases as the size of the input increases⁵⁶⁷⁸.

14. Explain Worst case, Best case and average case with example

Ans: - ****Worst case****: This refers to the most complex scenario for an algorithm. The worst case analysis considers the scenario in which the algorithm takes the longest possible time to complete its task. This is typically the scenario in which the input data is in a specific order or configuration that causes the algorithm to perform the maximum number of operations.

For example, for a linear search algorithm, the worst case occurs when the element being searched for is at the end of the list, resulting in a time complexity of $O(n)$.

- **Best case**: This refers to the simplest scenario for an algorithm.

The best case analysis considers the scenario in which the algorithm takes the shortest possible time to complete its task. This is typically the scenario in which the input data is in a specific order or configuration that causes the algorithm to perform the minimum number of operations.

For a linear search algorithm, the best case occurs when the element being searched for is at the beginning of the list, resulting in a time complexity of $O(1)$ ¹⁴.

- **Average case**: This refers to the average scenario for an algorithm.

The average case analysis considers the average time it takes the algorithm to complete its task over all possible input data. This is typically calculated by assuming that all input data is equally likely and then averaging the time it takes the algorithm to complete its task for each input. For a linear search algorithm, the average case would also result in a time complexity of $O(n)$, assuming that the element being searched for is equally likely to be at any position in the list¹⁴.

15. Write an algorithm to perform sequential search and also calculate its Worst case, Best case and average case complexity.

Ans: Sequential search is a simple algorithm for finding a specific element in a list of elements. It works by iterating through the list and comparing each element to the element to be searched for until the element is found or the end of the list is reached.

Algorithm in Python to perform a sequential search:

```
def sequential_search(lst, item):  
    for i in range(len(lst)):  
        if lst[i] == item:  
            return i  
    return -1
```

In this algorithm:

- The **worst-case** scenario is when the item is not in the list or is the last element in the list. The function would have to iterate through the entire list, resulting in a time complexity of $O(n)$ ¹⁵.
- The **best-case** scenario is when the item is the first element in the list. The function would find the item immediately, resulting in a time complexity of $O(1)$ ¹⁵.
- The **average-case** scenario, assuming that the item is equally likely to be at any position in the list, would result in a time complexity of $O(n)$ ¹⁵.
- The space complexity of this algorithm is $O(1)$ because it uses a constant amount of space.

16. Explain Big O notation with example.

Ans: In computer science, Big O Notation is a fundamental tool used to find out the time complexity of algorithms. Big O Notation allows programmers to classify algorithms depending on how their run time or space requirements vary as the input size varies.

Examples:

Runtime Complexity for Linear Search – $O(n)$

Runtime Complexity for Binary Search – $O(\log n)$

Runtime Complexity for Bubble Sort, Selection Sort, Insertion Sort, Bucket Sort - $O(n^2)$.

Runtime Complexity for Exponential algorithms like Tower of Hanoi - $O(2^n)$.

Runtime Complexity for Heap Sort, Merge Sort - $O(n \log n)$. Big O Notation gives the upper-bound runtime or worst-case complexity of an algorithm. It analyzes and classifies algorithms depending on their run time or space requirements.

17. Explain Big Omega notation with example.

Ans: Big Omega notation is used to describe the lower bound of an algorithm's time complexity, representing the best-case scenario¹. It provides a guarantee that the algorithm will not complete in less time than this bound¹.

For example, in the best-case scenario for a simple search algorithm (where the target value is the first element in the list), the algorithm runs in $\Omega(1)$ time, meaning it will take at least constant time¹.

18. Explain Big Theta notation with example.

Ans: Big Theta notation is used when the algorithm's upper and lower bounds are the same¹. It provides a tight bound on the time complexity of an algorithm¹.

For example, an algorithm that always takes a constant amount of time, regardless of the size of the input, is said to run in $\Theta(1)$ time¹. This means that the algorithm's time complexity is exactly constant, neither growing with the size of the input nor depending on the specific values of the input¹.

19. Explain asymptotic notations Big O, Big Ω and Big Θ that are used to compare the order of growth of an algorithm with example.

Ans: 1. Big O Notation (O-notation): Big O notation is used to describe the asymptotic upper bound, which provides an upper limit on the time complexity of an algorithm¹. It represents the worst-case scenario in terms of execution time¹. For example, if we have a simple search algorithm that checks each element of a list to find a target value, the worst-case scenario is that the target is at the end of the list or not in the list at all. In this case, the algorithm runs in $O(n)$ time, where n is the number of elements in the list¹.

2. Big Omega Notation (Ω -notation): Big Omega notation provides an asymptotic lower bound, which provides a lower limit on the time complexity of an algorithm¹. It represents the best-case scenario¹. For example, in the best-case scenario for a simple

search algorithm (where the target value is the first element in the list), the algorithm runs in $\Omega(1)$ time, meaning it will take at least constant time¹.

3. **Big Theta Notation (Θ -notation)**: Big Theta notation provides a tight bound on the time complexity of an algorithm¹. It is used when the algorithm's upper and lower bounds are the same¹. For example, an algorithm that always takes a constant amount of time, regardless of the size of the input, is said to run in $\Theta(1)$ time¹. This means that the algorithm's time complexity is exactly constant, neither growing with the size of the input nor depending on the specific values of the input¹. These notations are used to compare the order of growth of an algorithm as the size of the input increases¹. They provide a way to analyze an algorithm's running time by identifying its behavior as its input size grows¹.

20. Define Big O notation and prove

a) $100n + 5 = O(n^2)$

b) $5n^2 + 3n + 20 = O(n^2)$

c) $n^2 + n = O(n^3)$

d) $3n + 2 = O(n)$

e) $1000n^2 + 100n - 6 = O(n^2)$

Ans: **Big O notation** is a mathematical notation that describes the limiting behavior of a function when the argument tends towards a particular value or infinity⁶. In computer science, it is used to classify algorithms according to how their run time or space requirements grow as the input size grows⁶.

The formal definition is as follows: Let f and g be functions from the set of natural numbers to itself. The function f is said to be $O(g)$ (read big-oh of g), if there is a constant $c > 0$ and a natural number n_0 such that $f(n) \leq c \cdot g(n)$ for all $n \geq n_0$ ⁵.

a) **$100n + 5 = O(n^2)$** : We need to find constants c and n_0 such that $100n + 5 \leq c \cdot n^2$ for all $n \geq n_0$. If we choose $c = 105$ and $n_0 = 1$, then $100n + 5 \leq 105n^2$ for all $n \geq 1$ ⁹ [¹⁰].

b) **$5n^2 + 3n + 20 = O(n^2)$** : We need to find constants c and n_0 such that $5n^2 + 3n + 20 \leq c \cdot n^2$ for all $n \geq n_0$. If we choose $c = 10$ and $n_0 = 1$, then $5n^2 + 3n + 20 \leq 10n^2$ for all $n \geq 1$ ¹⁴.

c) **$n^2 + n = O(n^3)$** : We need to find constants c and n_0 such that $n^2 + n \leq c \cdot n^3$ for all $n \geq n_0$. If we choose $c = 1$ and $n_0 = 1$, then $n^2 + n \leq n^3$ for all $n \geq 1$ ³.

d) **$3n + 2 = O(n)$** : We need to find constants c and n_0 such that $3n + 2 \leq c \cdot n$ for all $n \geq n_0$. If we choose $c = 5$ and $n_0 = 1$, then $3n + 2 \leq 5n$ for all $n \geq 1$ ²¹.

e) **$1000n^2 + 100n - 6 = O(n^2)$** : We need to find constants c and n_0 such that $1000n^2 + 100n - 6 \leq c \cdot n^2$ for all $n \geq n_0$. If we choose $c = 2000$ and $n_0 = 1$, then $1000n^2 + 100n - 6 \leq 2000n^2$ for all $n \geq 1$ [¹⁰].

16. Define Big Omega notation and prove

a) $n^3 \in \Omega(n^2)$

b) Prove that $2n + 3 = \Omega(n)$

c) $\frac{1}{2}n(n-1) \in \Omega(n^2)$

d) Prove that $n^3 + 4n^2 = \Omega(n^2)$

Ans: **Big Omega (Ω) notation** is used to describe the lower bound of an algorithm's time complexity. It provides an asymptotic lower bound, guaranteeing that the algorithm will not perform faster beyond a certain input size.

a) $n^3 \in \Omega(n^2)$

This is true because n^3 grows at least as fast as n^2 for all $n > 1$. So we can choose $c = 1$ and $n_0 = 1$, and we have $0 \leq c \cdot g(n) \leq f(n)$ for all $n \geq n_0$.

b) $2n + 3 = \Omega(n)$

This is true because for $n > 3$, $2n + 3$ is always greater than n . So we can choose $c = 1$ and $n_0 = 3$, and we have $0 \leq c \cdot g(n) \leq f(n)$ for all $n \geq n_0$.

c) $\frac{1}{2}n(n-1) \in \Omega(n^2)$

This is true because $\frac{1}{2}n(n-1)$ simplifies to $\frac{1}{2}n^2 - \frac{1}{2}n$, and for $n > 2$, $\frac{1}{2}n^2 - \frac{1}{2}n$ is always greater than $\frac{1}{4}n^2$. So we can choose $c = \frac{1}{4}$ and $n_0 = 2$, and we have $0 \leq c \cdot g(n) \leq f(n)$ for all $n \geq n_0$.

d) $n^3 + 4n^2 = \Omega(n^2)$

This is true because $n^3 + 4n^2$ grows at least as fast as n^2 for all $n > 0$. So we can choose $c = 1$ and $n_0 = 0$, and we have $0 \leq c \cdot g(n) \leq f(n)$ for all $n \geq n_0$.

In each case, the function on the left grows at least as fast as the function on the right for n greater than n_0 , so the statement is true according to the definition of Big Omega notation.

17. Define Big Theta notation and prove

a) $n^2 + 5n + 7 = \Theta(n^2)$

b) $\frac{1}{2}n^2 + 3n = \Theta(n^2)$

c) $\frac{1}{2}n(n-1) \in \Theta(n^2)$

Ans: Big Theta (Θ) notation is used to describe the average-case performance of an algorithm. It provides an asymptotic tight bound, meaning it gives both an upper and a lower bound on the growth rate of runtime of an algorithm⁶⁷⁸⁹[¹⁰].

For a given function $g(n)$, we denote $f(n) \in \Theta(g(n))$ if there are positive constants c_1 , c_2 and n_0 such that $0 \leq c_1 \cdot g(n) \leq f(n) \leq c_2 \cdot g(n)$ for all $n \geq n_0$. This means $f(n)$ grows at the same rate as $g(n)$.

a) $n^2 + 5n + 7 = \Theta(n^2)$: This is true because for $n \geq 1$, $n^2 \leq n^2 + 5n + 7 \leq 13n^2$. So we can choose $c_1 = 1$, $c_2 = 13$, and $n_0 = 1$.

b) $\frac{1}{2}n^2 + 3n = \Theta(n^2)$: This is true because for $n \geq 6$, $\frac{1}{2}n^2 \leq \frac{1}{2}n^2 + 3n \leq 2n^2$. So we can choose $c_1 = \frac{1}{2}$, $c_2 = 2$, and $n_0 = 6$.

c) $\frac{1}{2}n(n-1) = \Theta(n^2)$: This is true because $\frac{1}{2}n(n-1)$ simplifies to $\frac{1}{2}n^2 - \frac{1}{2}n$, and for $n \geq 2$, $\frac{1}{2}n^2 - \frac{1}{2}n$ grows at the same rate as n^2 . So we can choose $c_1 = \frac{1}{2}$, $c_2 = 1$, and $n_0 = 2$.

In each case, the function on the left grows at the same rate as the function on the right for n greater than n_0 , so the statement is true according to the definition of Big Theta notation.

18. Explain with example mathematical analysis of non-recursive algorithm.

Ans: Mathematical analysis of non-recursive algorithms often involves determining the time complexity of the algorithm, which is a measure of the amount of time an algorithm takes to run as a function of the size of the input to the program. For example, consider a simple non-recursive algorithm that sums all the elements in an array:

Python'

```
def sum_array(arr):  
    total = 0  
    for num in arr:  
        total += num  
    return total
```

The time complexity of this algorithm is $O(n)$, where n is the number of elements in the array. This is because each operation (adding a number to the total) is performed n times.

19. Write an algorithm to Find the largest element in an array and also perform mathematical analysis.

Ans: Python code:

```
def find_largest(arr):  
    largest = arr[0]  
    for num in arr:  
        if num > largest:  
            largest = num  
    return largest
```

The time complexity of this algorithm is also $O(n)$, where n is the number of elements in the array. This is because we're comparing each element in the array to the current largest element once.

20. Write an algorithm to Checking for Unique elements in an array and also perform mathematical analysis.

Ans: Python'

```
def is_unique(arr):
    seen = set()
    for num in arr:
        if num in seen:
            return False
        seen.add(num)
    return True
```

The time complexity of this algorithm is $O(n)$, where n is the number of elements in the array. This is because we're checking and inserting each element in the set once.

21. Write an algorithm to perform matrix multiplication and also perform mathematical analysis.

Ans: algorithm to perform matrix multiplication:

```
```python
def matrix_multiply(A, B):
 n = len(A)
 C = [[0 for _ in range(n)] for _ in range(n)]
 for i in range(n):
 for j in range(n):
 for k in range(n):
 C[i][j] += A[i][k] * B[k][j]
 return C
```
```

This algorithm assumes that 'A' and 'B' are square matrices of the same size. The time complexity of this algorithm is $O(n^3)$, where n is the number of rows (or columns) in each matrix. This is because there are three nested loops, each of which runs 'n' times. The time complexity depends on the size of the input and the specific operations performed in the algorithm.

22. Write a non-recursive algorithm to Count the number of bits in a number. And also perform mathematical analysis.

Ans: Python code:

```
def count_bits(n):
    count = 0
    while n:
        count += n & 1
        n >>= 1
    return count
```

This algorithm uses bitwise operations to count the number of bits in `n`. The time complexity of this algorithm is $O(\log n)$, where n is the number being examined. This is because the number of bits in `n` is proportional to the logarithm (base 2) of `n`. The time complexity depends on the size of the input and the specific operations performed in the algorithm.

23. List the steps for analyzing the time efficiency of recursive algorithm.

Ans: Steps for Analyzing the Time Efficiency of Recursive Algorithm:

1. Define the Recurrence Relation: Identify the recursive relationship that describes how the problem is decomposed into smaller subproblems. This recurrence relation is often a key to understanding the algorithm's time complexity.
2. Write Down the Base Case(s): Identify the base case(s) of the recursion, which represent the smallest subproblems that can be directly solved. Base cases are crucial for determining when the recursion stops.
3. Determine the Size of Subproblems: Analyze how the size of the problem decreases with each recursive call. Define the size of the subproblems in terms of the input size.
4. Express the Recurrence Relation: Write down the recurrence relation explicitly, expressing the time complexity of the algorithm in terms of the size of the input and the time complexity of smaller subproblems.
5. Solve or Simplify the Recurrence: Solve the recurrence relation or simplify it to obtain a closed-form expression. This step involves expressing the time complexity of the algorithm as a function of the input size without recursion.
6. Determine the Dominant Term: Identify the dominant term in the closed-form expression. The dominant term indicates the growth rate of the time complexity and is often the most significant factor in determining the overall efficiency.
7. Analyze the Time Complexity: Express the time complexity of the recursive algorithm using Big O notation. The result should provide a clear understanding of how the algorithm's efficiency scales with the input size.
8. Verify with Examples: Validate the derived time complexity by running the algorithm on different inputs and comparing the observed behavior with the theoretical analysis.
9. Consider Best, Worst, and Average Cases: Analyze the best-case, worst-case, and average-case time complexities separately, if applicable. Different input scenarios may lead to distinct time complexities.
10. Optimize if Necessary: If the derived time complexity is not acceptable, consider optimizing the algorithm or exploring alternative algorithmic approaches to achieve the desired efficiency.

By following these steps, you can systematically analyze the time efficiency of a recursive algorithm, gaining insights into its performance characteristics and enabling informed decisions about its suitability for specific use cases.

23. Explain with example mathematical analysis of recursive algorithm.

Ans: Let's consider the recursive algorithm for factorial calculation:

Python

```
def factorial(n):  
    if n == 0:  
        return 1  
    else:  
        return n * factorial(n-1)
```

The base case is $n == 0$, where the function returns 1. The recurrence relation can be defined as $T(n) = T(n-1) + c$, where c is a constant representing the time to perform the multiplication. This relation says that the time to compute $\text{factorial}(n)$ is the time to compute $\text{factorial}(n-1)$ plus the time for the multiplication operation.

Solving this recurrence relation gives $T(n) = cn$, which means the time complexity of the algorithm is $O(n)$.

24. Write an algorithm to find the factorial of a number using recursion and also perform mathematical analysis.

Ans: Recursive Algorithm for Factorial Calculation:

Python'

```
def factorial(n):  
    if n == 0:  
        return 1  
    else:  
        return n * factorial(n - 1)
```

Mathematical Analysis:

The factorial of a non-negative integer n is the product of all positive integers from 1 to n . Mathematically, it is represented as:

$$n! = n * (n - 1) * (n - 2) * \dots * 1$$

Using the recursive definition of factorial, we can derive the following recurrence relation:

$$n! = n * (n - 1)!$$

This recurrence relation forms the basis of the recursive algorithm for calculating factorial. The base case of the recursion is $n = 0$, for which $\text{factorial}(0) = 1$.

25. Write an algorithm to perform Towers of Hanoi using recursion and also perform mathematical analysis.

Ans: Towers of Hanoi Algorithm Using Recursion:

Python algorithm to solve the Towers of Hanoi problem using recursion:

```
def hanoi(n, source, target, auxiliary):
```

```
    if n > 0:
```

```
        # Move n - 1 disks from source to auxiliary, so they are out of the way
```

```
        hanoi(n - 1, source, auxiliary, target)
```

```
        # Move the nth disk from source to target
```

```
        print('Move disk', n, 'from peg', source, 'to peg', target)
```

```
        # Move the n - 1 disks that we left on auxiliary to target
```

```
        hanoi(n - 1, auxiliary, target, source)
```

```
    # Test the function
```

```
    hanoi(3, 'A', 'C', 'B')
```

In this algorithm, n is the number of disks, and source, target, and auxiliary are the pegs. The base case is when $n == 0$, and it does nothing. For other cases, it makes two recursive calls with $n-1$.

The time complexity of this algorithm is $O(2^n)$, because each call to hanoi results in two more calls. The space complexity is $O(n)$, because in the worst case, if a recursive call is made n times, there will be n activation records on the call stack.

26. State the recursive algorithm to count the bits of a decimal number in its binary representation. Give its mathematical analysis.

Ans: Counting the Bits of a Decimal Number in Its Binary Representation:

Here's a Python algorithm to count the bits of a decimal number in its binary representation using recursion:

```
def count_bits(n):
```

```
    if n == 0:
```

```
        return 0
```

```
    else:
```

```
        return 1 + count_bits(n & (n - 1))
```

In this algorithm, the base case is when $n == 0$, and it returns 0. For other cases, it makes a recursive call with $n \& (n - 1)$, which has the effect of 'turning off' the rightmost '1' bit in n , and adds 1 to the count.

The time complexity of this algorithm is $O(\log n)$, because each recursive call reduces the number of '1' bits in n by one, and there can be at most $\log n$ '1' bits in n . The space complexity is also $O(\log n)$, because in the worst case, if a recursive call is made $\log n$ times, there will be $\log n$ activation records on the call stack.

27. Consider the following algorithm.

GUESS[A][[]]

for i<-0 to n-1

for j<-0 to i

A[i][j]<-0

i) What does the algorithm compute?

ii) What is its basic operation?

iii) What is the efficiency of this algorithm?

Ans: i) The algorithm 'GUESS[A][[]]' initializes the lower triangle of a square matrix 'A' (including the diagonal) with zeros. The lower triangle of a matrix includes all the elements 'A[i][j]' where 'i >= j'.

ii) The basic operation of this algorithm is the assignment operation 'A[i][j] <- 0'.

iii) The efficiency of this algorithm can be analyzed as follows:

The outer loop runs 'n' times, and for each iteration 'i' of the outer loop, the inner loop runs 'i+1' times. Therefore, the total number of operations is '1 + 2 + 3 + ... + n = n*(n+1)/2'.

In terms of time complexity, we ignore lower order terms and multiplicative constants for large 'n', so this simplifies to $O(n^2)$. This means that the time complexity of the algorithm is quadratic in the size of the input 'n'. The space complexity of the algorithm is $O(1)$, which means the amount of space used does not change with the size of the input, because it only uses a constant amount of space to store the indices and the matrix remains the same size.

27. Solve the following recurrence relation.

a) $x(n) = x(n-1) + 5$ for $n > 1$, $x(1) = 0$

b) $x(n) = 3x(n-1)$ for $n > 1$, $x(1) = 4$

c) $x(n) = x(n-1) + n$ for $n > 1$, $x(0) = 0$

Ans: a) The recurrence relation is given by: ' $x(n) = x(n-1) + 5$ ' for ' $n > 1$ ', ' $x(1) = 0$ '. To find ' $x(n)$ ', we repeatedly substitute the recurrence into itself. For example, ' $x(2) = x(2-1) + 5 = 0 + 5 = 5$ ', ' $x(3) = x(3-1) + 5 = 5 + 5 = 10$ '. So, we can observe a pattern and the general solution is ' $x(n) = 5 * (n-1)^1$ '.

b) The recurrence relation is given by: ' $x(n) = 3x(n-1)$ ' for ' $n > 1$ ', ' $x(1) = 4$ '. Similarly, we find the values for ' $x(n)$ ' iteratively. For example, ' $x(2) = 3 * x(2-1) = 3 * 4 = 12$ ', ' $x(3) = 3 * x(3-1) = 3 * 12 = 36$ '. So, the general solution is ' $x(n) = 4 * 3^{(n-1)^1}$ '.

c) The recurrence relation is given by: ' $x(n) = x(n-1) + n$ ' for ' $n > 0$ ', ' $x(0) = 0$ '. Here, each term is the sum of the previous term and 'n'. For example, ' $x(1) = x(1-1) + 1 = 0 + 1 = 1$ ', ' $x(2) = x(2-1) + 2 = 1 + 2 = 3$ '. From this, the general solution ' $x(n) = n * (n+1) / 2^1$ '.

28. Find the time complexity of below algorithm:

Fibonacci(n)

```
{  
if n<=1 then  
output 'n'  
else  
f2=0;  
f1=1;  
for i=2 to n do  
{  
f=f1+f2;  
f2=f1;  
f1=f;  
}  
output 'f'  
}
```

Ans: Analysis of the algorithm:

- The algorithm has a conditional statement checking whether (n) is less than or equal to 1. If true, it directly outputs (n) .
- Otherwise, it initializes two variables $(f2)$ and $(f1)$ to 0 and 1, respectively.
- It then iterates through a loop $(n-1)$ times, updating (f) in each iteration.
- The loop has constant time operations inside it (additions and assignments).

****Time Complexity:****

- The loop iterates $(n-1)$ times, resulting in a linear dependence on (n) .
- The constant time operations inside the loop do not affect the overall time complexity.

Therefore, the time complexity of this algorithm is $(O(n))$, where (n) is the input to the Fibonacci function.